

Project 2 Readme Team esn

Version 1 9/11/24

1	Team Name: esn														
2	Team members names and netids: Emily Norton - enorton3														
3	Overall project attempted, with sub-projects: NTM Tracer														
4	Overall success of the project: Successful - I was able to successfully trace the NTM, keeping track of all possible choices in a tree and reconstructing the actual path taken for accepted strings.														
5	Approximately total time (in hours) to complete: 6 hours														
6	Link to github repository: https://github.com/emillynorton/Project2-TOC														
7	List of included files (if you have many files of a certain type, such as test files of different sizes, list just the folder): (Add more rows as necessary). Add more rows as necessary.<table><tr><th>File/folder Name</th><th>File Contents and Use</th></tr><tr><td colspan="2">Code Files</td></tr><tr><td>ntm_tracer.py</td><td>Contains run() and print_trace_path() functions to trace an input through a nondeterministic turing machine, determine if an accept/reject state is reached, and backtrack to print the path taken.</td></tr><tr><td colspan="2">Test Files</td></tr><tr><td>abc.csv aplus.csv composite.csv equal_01s.csv ntm_n1n.csv</td><td>All files define different nondeterministic turing machines and are used to test the ntm_tracer program.</td></tr><tr><td colspan="2">Output Files</td></tr><tr><td>aplus-aaa-esn.png aplus-aaaaa-esn.png</td><td>The output is printed to the terminal (as directed</td></tr></table>	File/folder Name	File Contents and Use	Code Files		ntm_tracer.py	Contains run() and print_trace_path() functions to trace an input through a nondeterministic turing machine, determine if an accept/reject state is reached, and backtrack to print the path taken.	Test Files		abc.csv aplus.csv composite.csv equal_01s.csv ntm_n1n.csv	All files define different nondeterministic turing machines and are used to test the ntm_tracer program.	Output Files		aplus-aaa-esn.png aplus-aaaaa-esn.png	The output is printed to the terminal (as directed
File/folder Name	File Contents and Use														
Code Files															
ntm_tracer.py	Contains run() and print_trace_path() functions to trace an input through a nondeterministic turing machine, determine if an accept/reject state is reached, and backtrack to print the path taken.														
Test Files															
abc.csv aplus.csv composite.csv equal_01s.csv ntm_n1n.csv	All files define different nondeterministic turing machines and are used to test the ntm_tracer program.														
Output Files															
aplus-aaa-esn.png aplus-aaaaa-esn.png	The output is printed to the terminal (as directed														

	composite-111111-esn.png equal01-011010-esn.png equal01-10110-esn.png in the instructions). The output files contain screenshots of what was printed to the terminal to prove the functionality of the NTM tracer program.
	Plots (as needed)
	Not applicable
8	Programming languages used, and associated libraries: I used python. I did not load any additional libraries beyond what was included in the starter code. The csv and sys libraries were used in the turing_machine.py starter code.
9	Key data structures (for each sub-project): - Lists (lists of 3 to represent configurations in each machine and lists of lists to build trees) - Dictionaries (to define read symbol, next state, write symbol, and move for each transition on the machine)
10	General operation of code (for each subproject): The ntm_tracer.py file has two key functions: run and print_trace_path. In the run function, I used two trees. The first is a list, which contains sublists for each level of the tree, that also contain sublists that represent possible configurations at that level (formatted [left, state, right]). The second is an extended tree that stores the same levels and configurations, but also stores the index of the parent and the number of the transition that was taken inside each configuration list. Storing the parent index and transition number made it easier to backtrack through the tree and reconstruct the path that the NTM took to the accept state. The run function executes a breadth-first exploration of all possible paths that the NTM can take. It begins by adding the initial configuration to both trees, then it follows the transitions to append configurations of possible paths to both trees. Every time a configuration is added, it checks if an accept/reject state is reached. If not, it creates all the valid transitions based on the transition function, and adjusts the tape according to the transition. If the string is accepted, the tree is printed and print_trace_path is called to reconstruct the path. The string is rejected if there are no valid transitions from the current state, or if the transition goes to the reject state. The function stops when the string is accepted, rejected, or the search reaches the max depth. The print_trace_path function reconstructs and prints the path that the NTM took from the initial configuration to the accepting configuration. It traces the tree up from the final

	node, using the parent index stored in the extended tree to determine which node preceded the current node. After walking up the entire tree, the path is reversed and printed from root node to accepting branch.
11	What test cases you used/added, why you used them, what did they tell you about the correctness of your code. abc.csv - I added this input as a simple test case that I could use to visualize the tree expanding and follow the input "abc" to make sure that my ntm was building a proper tree. This machine was not designed to look for a specific pattern, I merely manually added rules so that I could watch the tree expand and check that the left/right partitions were correct for each expansion. This was very helpful in checking the functionality of my code. aplus.csv - This input file was provided. I checked that the input "aaaaa" was accepted at a depth of 6, as outlined in the instructions. composite.csv - I got this input file from the test files spreadsheet shared with the class. It accepts $\{1^n \mid n \text{ is composite}\}$. I verified that my function rejects inputs 1, 11, and 111, but accepts inputs 111111 and 111111111. equal_01s.csv - I also got this file from the class test files spreadsheet. I used it to verify that my ntm accepts strings with equal 0/1's and rejects strings that do not meet this condition. ntm_n1n.csv - I used this input file as it was provided in the Github starter code, but I was a bit confused about its purpose. I thought it was supposed to accept all strings of the form $0^n 1^n$ (based on the machine name), but it rejected 01. I traced the input 01 by hand and discovered that it should be rejected according to the rules of the machine. This helped me verify that my tree was correct.
12	How you managed the code development: I completed my implementation of the ntm_tracer, and tested it on numerous input files. I then pushed all my changes to the main branch of my Github. In doing some reflection, I realized that I probably could have made a branch to record my work and pushed to it more consistently. Then, I could have merged my branch with main. However, since I was working individually, it was easy to manage my Github changes without creating proper branches. After that, I pushed to Github occasionally as I came across improvements that I could make to my ntm tracer and experimented with new input files.
13	Detailed discussion of results: My ntm tracer works very well. The results that it produces match all cases that I have tested by hand - particularly the ones that I tested with the abc.csv and the aplus.csv. The tree produced exactly what I expected and the paths that it builds are correct. The output also matches what is expected for the composite.csv (strings with a composite number of 1's are accepted and others are rejected) and the equal_01s.csv (strings with

	equal 0/1's are accepted and others are rejected).
14	How team was organized: I worked individually, so it was easy to manage the work associated with the project. I worked consistently for a few days, and was able to finish before the deadline without stress.
15	What you might do differently if you did the project again: I need to get better about pushing my changes to Github more consistently. I tend to continue working until I am essentially done and then push my work. However, this makes it difficult if I ever need to go back to previous versions of the code. If I pushed changes consistently, then I would have a more complete version history to reference if necessary. I should also get better at using well-named branches to distinguish between the changes that I am making, so I can reference the code for a specific feature.
16	Any additional material: None